

Triage System Manual

How to run the system, what happens to a complaint between typing it and reading a triage level, and the four things it will not tell you.

BEFORE YOU START

Two services must be running

The system translates every complaint on this machine before it scores it. Nothing reaches the internet — but nothing works without the local translator either.

```
ollama serve          # the local translator
ollama pull llama3.2  # once, ~2 GB
./run_gui.sh          # the app
```

Ollama does not always survive a reboot. If it keeps stopping:

```
sudo systemctl enable --now ollama
```

To check everything the pipeline needs without running a patient through it:

```
python run_inference.py --check
```

TRIAGING ONE PATIENT

The everyday task

1 Type the complaint in Roman Urdu

Spelling does not need to be consistent. `bukar`, `payt` and `darad` are repaired before the translator sees them.

2 Fill in the vitals and the four dropdowns

A blank or unreadable vital is replaced with the training average, and the substitution is written into the notes. It is never silent.

3 Press "Triage this patient"

Translation takes about 11 seconds. The stage panel on the right shows all five steps, so you can see what the system did rather than only what it concluded.

4 Read the level, then read the confidence

They are different claims. See *Reading the output* on page 3.

TRIAGING A FILE

Batch runs are slow by design

One local translation per row, roughly 11 seconds each. A 100-patient sheet takes about **18 minutes**; 500 takes an hour and a half. The progress bar shows the row count and an estimate — it makes the wait legible, not shorter.

Rows the system could not translate, or whose translation failed the anatomical check, are **not scored**. They appear in the table with a gate status and no triage level, and the summary line states how many were skipped. A run that quietly dropped them would read as a complete triage of the file.

Five stages, and where each one can fail

Every stage runs on this machine. Ollama is a service on `127.0.0.1`, the encoder loads from an on-disk cache, and the classifier is a file in the model folder.

1 Dictionary and fuzzy repair — local, instant

Spelling variants collapse onto one canonical word: `payt` and `paet` both become `pait`. Typos are repaired only when they are close to a known word and long enough to be a typo rather than a different word — a deliberately cautious rule, because a careless version of it once turned `peene` ("to drink") into `seene` ("chest").

2 Translation by Ollama — ~11 s

The complaint is sent to llama3.2 wrapped as a data record, at temperature 0 so the same input always gives the same English. If the model replies with a refusal or a safety disclaimer, that text is stripped and the row is retried once. A disclaimer that reached the encoder would be scored as *if it were the patient's symptoms*.

3 Anatomical check — this is the safety gate

Every body part named in the complaint must appear in the English. A stomach complaint rendered as chest pain is refused outright — no triage level is produced. This check is deterministic: it either finds the word or it does not.

4 Encoding

Common English words carrying no clinical signal are removed, and the remaining text becomes a 384-number vector.

5 Classification

The vector, the six vitals and the four categories go to a random forest, which votes on a triage level.

WHY THE ANATOMICAL CHECK, AND NOT A SIMILARITY SCORE

A measurement that changed the design

The system used to compare the Roman Urdu and English texts numerically and reject the translation if they were too far apart. Measured against `seena mein shadeed dard aur pasina` (severe chest pain with sweating):

CANDIDATE TRANSLATION	SIMILARITY	VERDICT
"Severe chest pain and sweating."	0.8054	correct
"I have a headache and dizziness."	0.7908	wrong organ
"My leg is broken after a fall."	0.7922	unrelated

Correct and catastrophically wrong sit **0.013 apart**, and the unrelated sentence scores *higher* than the merely wrong one. No cut-off separates them. The similarity number is still calculated and displayed, and it decides nothing.

The triage level

LEVEL	MEANING	TARGET
1	Emergency	immediate
2	Urgent	within 15 minutes
3	Standard	within 60 minutes
4	Non-urgent	can wait or be redirected

The confidence number

Confidence is the share of trees in the forest that voted for the winning level. It is **not calibrated**: 76% does not mean this patient is Level 1 in 76% of comparable cases. Use it to compare one prediction against another, never as a probability to act on numerically.

When a complaint carries no usable text — blank, n/a, punctuation only — the confidence is capped at **50%** and a warning says why. The vital signs alone can push the forest above 90%, and a screen showing that would be indistinguishable from a real reading. Where the cap applies, the bar chart labels both numbers so the raw vote and the reported figure never silently disagree.

WHAT THE TABS ARE FOR

TAB	USE IT WHEN
Triage a Patient	Scoring one patient. The everyday screen.
Pipeline Explorer	A result looks wrong and you want to see which stage caused it.
Stop Words	Checking which English words are discarded before encoding, and the statistics behind each decision.
Batch File	Scoring a spreadsheet of patients.
Results	Precision, recall and F1 for the model currently serving.
Cluster Analysis	Checking whether complaints that mean the same thing land near each other.

WHEN SOMETHING GOES WRONG

WHAT YOU SEE	WHAT IT MEANS
"Ollama is not reachable"	The translator is not running. <code>ollama serve</code> . Nothing can be scored until it is.
"returned nothing"	The model refused twice. The console prints its exact reply.
"Anatomical check failed"	The translation moved the complaint to a different body part. No prediction was made — this is the gate working.
Translation timeout	Cold start under load. Re-run it; the second attempt is usually immediate.
Batch row marked NOT TRANSLATED	That row alone was skipped. The rest of the file is scored.

Do not start a batch while the app is loading its first prediction

Both load models onto the same CPU, and the translator's cold start can then exceed its timeout — producing a translation failure for a service that is running perfectly well. It is the single most common false alarm in this system.

LIMITS

Four things this system will not tell you

Each of these is measured, not hypothetical. A reader who takes only one page from this manual should take this one.

1 The training data is synthetic

The deployed model was fitted on complaints produced by a generator script, not on real patient records. It must not be described as trained on real cases in any evaluation, publication or deployment.

2 The gate checks anatomy, not accuracy

It guarantees the body part survives translation. It cannot see a wrong *symptom* attached to the right body part — "chest pain" rendered as "chest burning" passes. It is one specific guarantee, not a general one.

3 The serving model was trained on 2,252 rows, not 10,000

A larger Roman Urdu model exists in the project but expects untranslated text, so it cannot be fed the English the app now produces. The app serves the smaller English-trained model. This was a deliberate trade and it is a real reduction in training data.

4 The spelling repair is unproven on real input

Across the 10,000-row training corpus it changes nothing, because that corpus was generated from clean text and has no typos to repair. What has been demonstrated is that it corrupts nothing. Its benefit on real typed input is expected, not measured.

STATUS

Research prototype — not a medical device

This system has not been clinically validated, registered, or approved for use in patient care. It is a decision-support prototype for research. Every output is a suggestion to a qualified clinician, and no output should be acted on without one.

VERIFICATION

Two test batteries ship with the project and can be re-run at any time. Both need a live Ollama; there is no continuous integration, so they are run by hand.

```
python tests/audit_pipeline.py    # 15 checks: prompt, guardrail,  
                                  # gate, dictionary, train/serve skew  
python tests/audit_gui.py         # 16 checks: builds the real window  
                                  # and drives every tab
```

Figures in this manual — the 0.013 similarity gap, the 11-second translation, the 50% cap — were measured on the running system, not estimated.